IB Comp Sci Notes

Kenny Lin / BusyPizza

November 3, 2026

Abstract

This guide / note is dedicated to the love of my life: computer science. Wish y'all have a great time studying this subject! (第一页用了纯黑背景其实是为了配合下面的插图，后面是正常的白底黑字)
Notes to Mr. Sanchez: this note is for my own use (and I might share this with classmates later this year once it reaches high level of completion), therefore you might see Chinese in it. Simply ignore it.



Contents

A Concepts of Computer Science **4**
A1 Computer fundamentals . . . 4
A1.1 Computer hardware and operation . . . 4
A1.1.1 Functions and interactions of the main CPU components . . . 4
A1.1.2 Role of a GPU . . . 5
A1.1.3 Differences between the CPU and GPU (HL only) . . . 6
A1.1.4 Purposes of different types of primary memory . . . 6
A1.1.5 The fetch, decode and execute cycle . . . 7
A1.1.6 Pipelining in multi-core architectures (HL only) . . . 7
A1.1.7 Internal and external secondary memory storage . . . 7
A1.1.8 Compression . . . 8
A1.1.9 Services in cloud computing . . . 9
A1.2 Data representation and computer logic . . . 10
A1.2.1 Principal methods of representing data . . . 10
A1.2.2 How binary is used to store data . . . 10
A1.2.3 Purpose and use of logic gates . . . 12
A1.2.4 Constructing and analysing truth tables . . . 12
A1.2.5 Constructing logic diagrams . . . 14
A1.3 Operating systems and control systems . . . 15
A1.3.1 Role of operating systems . . . 15
A1.3.2 Functions of an operating system . . . 15
A1.3.3 Comparing approaches to scheduling . . . 16
A1.3.4 Evaluating polling and interrupt handling . . . 16
A1.3.5 Multitasking and resource allocation (HL only) . . . 17
A1.3.6 Control system components (HL only) . . . 18
A1.3.7 Real-world uses of control systems (HL only) . . . 19
A1.4 Translation (HL only) . . . 20
A1.4.1 Evaluating interpreters and compilers . . . 20
A2 Networks . . . 22
A2.1 Network fundamentals . . . 22
A2.2 Network architecture . . . 22
A2.3 Data transmission . . . 22
A2.4 Network security . . . 22
A3 Databases . . . 22
A3.1 Database fundamentals . . . 22
A3.2 Database design . . . 22
A3.3 Database programming . . . 22
A3.4 Alternative databases and data warehouses . . . 22
A4 Machine learning . . . 22
A4.1 Machine learning fundamentals . . . 22
A4.2 Data preprocessing . . . 22
A4.3 Machine learning approaches . . . 22
A4.4 Ethical considerations . . . 22

B Computational Thinking and Problem Solving **23**
B1 Computational thinking . . . 23
B1.1 Approaches to computational thinking . . . 23
B1.1.1 Problem specification . . . 23
B1.1.2 Fundamental concepts of computational thinking . . . 25

B1.1.3 Applying computational thinking to solve problems . . . 25
B1.1.4 Tracing flowcharts for programming algorithms . . . 27
B2 Programming . . . 30
B2.1 Programming fundamentals . . . 30
B2.2 Programming constructs . . . 30
B2.3 Data structures . . . 30
B2.4 Programming algorithms . . . 30
B2.5 File processing . . . 30
B3 Object-oriented programming (OOP) . . . 30
B3.1 Fundamentals of OOP for a single class . . . 30
B3.2 Fundamentals of OOP for multiple classes . . . 30
B4 Abstract data types (ADTs) . . . 30
B4.1 Fundamentals of abstract data types . . . 30

C Appendices **31**
C0.1 My favourite operating system: Arch Linux (+ Hyprland) . . . 31
C0.1.1 What it is . . . 31
C0.1.2 Why it is my favourite . . . 31

A Concepts of Computer Science

A1 Computer fundamentals

Revision scope: Oxford, MacKenty and Stephenson (2025), A1.1–A1.4; checked against the IB Computer Science guide (first assessment 2027). Each numbered paragraph matches a syllabus statement. Content marked HL only is additional higher level material. Explanations and worked examples focus on the specified knowledge and skills.

A1.1 Computer hardware and operation

A1.1.1 Functions and interactions of the main CPU components

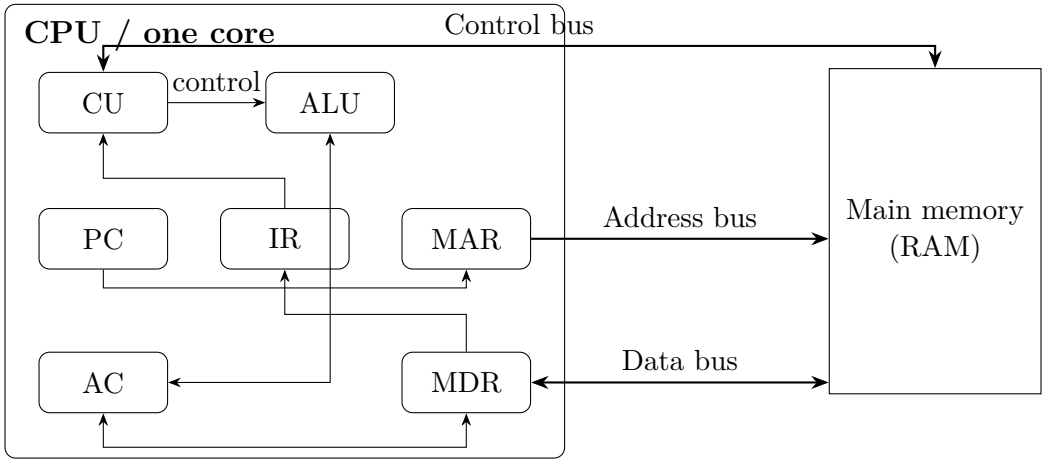
The **central processing unit (CPU)** executes machine-language instructions and coordinates the movement and processing of data. Its components cooperate during the fetch–decode–execute cycle.

- **Arithmetic logic unit (ALU)**: performs arithmetic, logical operations and comparisons on operands, producing results that can be placed in registers.
- **Control unit (CU)**: decodes instructions and generates control signals to coordinate registers, the ALU, memory and input/output (I/O). It controls the sequence and timing of operations.
- **Registers**: very small, fast storage locations inside a core, holding the instructions, addresses and data currently needed.

Register	Role
Instruction register (IR)	Holds the current instruction while it is decoded and executed; also called the current instruction register (CIR).
Program counter (PC)	Holds the address of the next instruction to fetch. Normally advances to the next instruction; a branch can replace its value.
Memory address register (MAR)	Holds the address of the memory location being read or written.
Memory data register (MDR)	Holds the instruction or data being transferred to or from memory.
Accumulator (AC/ACC)	Holds operands and intermediate results of ALU operations in an accumulator-based model.

A **bus** is a communication pathway between components. Distinguish the information carried from its destination.

- **Address bus**: carries the address of a location to read or write. In the simplified CPU–memory model, addresses travel from CPU to memory (unidirectional). Its width limits the number of distinct locations that can be addressed: n address bits allow 2^n locations.
- **Data bus**: carries data and instructions in either direction between CPU, memory and I/O (bidirectional). A wider bus can transfer more bits per transfer, other factors being equal.
- **Control bus**: carries signals such as memory read/write, interrupt requests and acknowledgements. Different signals travel in different directions, so the bus as a whole supports communication both ways.



Simplified functional diagram: arrows inside the core show selected register transfers and control relationships, not every physical connection. The PC supplies an address to the MAR; a fetched instruction passes through the MDR to the IR.

Processor types: a **single-core** CPU has one core; it can share time among tasks but cannot run separate threads on separate cores. A **multi-core** CPU has two or more cores, each able to execute a thread independently, enabling parallel execution. Speed-up depends on how much work can run independently and on communication and shared-memory costs. A **co-processor** assists the CPU with specialized work, such as graphics (GPU) or signal processing (DSP), freeing CPU time for general work.

A1.1.2 Role of a GPU

A **graphics processing unit (GPU)** accelerates graphics rendering and other suitable computations. Its many arithmetic processing units execute similar operations on many data items in parallel. This gives **high throughput**: a large amount of work completed per unit time. High memory bandwidth supplies the data these units need. A GPU may be integrated and share system RAM, or discrete with its own graphics memory (VRAM).

- **Video games and graphics**: calculate geometry, lighting, shading, textures and pixel colours for many parts of a scene, producing frames quickly enough for interactive display.
- **AI and machine learning**: accelerate matrix and vector operations used in training and running neural networks. Many elements can be calculated independently.
- **Large simulations**: perform repeated calculations across particles or grid cells, for example in weather or fluid simulations.
- **Video editing and rendering**: accelerate image effects, rendering and supported video-processing operations.

A GPU is most useful when the workload has enough parallel work; a small, sequential or heavily branching task may benefit little because coordination and data transfer also take time.

A1.1.3 Differences between the CPU and GPU (HL only)

Aspect	CPU	GPU
Design philosophy	Flexible general-purpose processing and low latency for individual tasks.	High throughput for many similar operations on large data sets.
Core architecture	Fewer, complex cores; sophisticated control flow and branch handling; substantial caches.	Many arithmetic units organized for parallel work; less suited to divergent control flow. CPU and GPU "core" counts are not directly comparable.
Processing power	Effective for sequential, decision-heavy work, OS services and coordinating applications.	Effective for large parallel numerical workloads and rendering; cannot be ranked as faster for every task.
Memory access	Cache hierarchy reduces latency when accessing frequently used data in RAM.	High bandwidth feeds parallel execution units. Discrete GPUs commonly use VRAM; shared-memory designs also exist.
Power efficiency	Often more efficient for small or sequential workloads.	Can perform more suitable parallel work per joule, even while drawing more total power. Efficiency depends on workload and transfer overhead.

Working together: the CPU handles program control, input and task preparation; it sends suitable parallel work to the GPU. With a discrete GPU, input data may be copied from RAM to VRAM and results copied back, adding delay. Shared-memory systems reduce some transfer needs, but still need coordination. The CPU launches a GPU task and must wait or receive a completion signal before using its results. For a game, the CPU processes player input and game rules while the GPU renders the scene. For ML, the CPU prepares batches while the GPU calculates large matrix operations.

A1.1.4 Purposes of different types of primary memory

Type	Volatility	Purpose and interaction with the CPU
Registers	Volatile	Smallest and fastest working storage inside a core; holds the current instruction, addresses, operands and results.
Cache	Volatile	Stores copies of recently/frequently used instructions and data, reducing access to slower RAM. L1 is typically smallest and fastest, followed by L2, then L3. L1 and often L2 are associated with individual cores; L3 is often shared. Layout varies.
RAM	Volatile	Holds the OS, active programs and working data. Larger than cache, but slower. Programs/data are loaded from secondary storage into RAM for execution. Unsaved contents are lost when power is removed.
ROM	Non-volatile	Holds firmware used for hardware initialization and starting the boot process. Traditionally read-only; modern firmware is often stored in flash that can be updated using a controlled process.

Performance hierarchy: registers → L1 → L2 → L3 → RAM generally means increasing capacity and access time, with decreasing cost per bit. Cache is normally on the processor chip; L3 is not defined as being off-chip. ROM serves a different purpose and is not another level of the working-data cache hierarchy.

A **cache hit** occurs when the requested instruction/data is found in the cache being checked. A **cache miss** means it is absent from that level; the request must be served by another level or RAM, adding delay. An L1 miss can still be an L2 hit. Cache hardware keeps useful copies nearby and replaces entries as needed. Repeated access to the same data, or nearby addresses, makes caching effective. A larger cache may help, but performance also depends on the access pattern; more cache is not a guarantee of proportional speed-up.

A1.1.5 The fetch, decode and execute cycle

A **machine-language instruction** is a binary-encoded command understood by a processor. Its **opcode** specifies the operation; any operands identify data, registers or addresses. The instruction cycle repeats while a program runs.

1. **Fetch**: copy the next instruction's address from PC to MAR. Send the address along the address bus and issue a memory-read signal on the control bus. Memory returns the instruction along the data bus into MDR; copy it into IR. Advance PC to the next instruction's address. In a simplified fixed-size instruction model this is $PC \leftarrow PC + 1$; the actual increment depends on instruction size and addressing.

2. **Decode**: the CU interprets the opcode and identifies the operands and required operations. Operand data may be retrieved from registers or through another memory read using MAR/MDR.

3. **Execute**: control signals cause the operation to take place. The ALU may calculate a result into AC or another register; a load/store transfers data; a branch changes PC. A store uses MAR for the destination address, MDR for the value, the data bus for the transfer and a write signal on the control bus.

Worked trace (simplified accumulator machine): initially $PC = 20$, $AC = 5$, memory location 20 contains `ADD 80`, and location 80 contains 7. Fetch: $MAR \leftarrow 20$, $MDR \leftarrow$ `ADD 80`, $IR \leftarrow MDR$, $PC \leftarrow 21$. Decode: identify addition with the value at address 80. Execute: $MAR \leftarrow 80$, $MDR \leftarrow 7$ through a memory read, then $AC \leftarrow 5 + 7 = 12$. Location 80 remains 7. The PC contains an *address*; the IR contains an *instruction*.

A1.1.6 Pipelining in multi-core architectures (HL only)

Pipelining overlaps stages of different instructions within a core. In a simplified four-stage pipeline: **fetch** retrieves the instruction; **decode** identifies the operation and operands; **execute** performs the operation; **write-back** records its result in the destination register (or memory, where required by the model).

Instruction	Cycle 1	2	3	4	5	6	7
I_1	F	D	E	W			
I_2		F	D	E	W		
I_3			F	D	E	W	
I_4				F	D	E	W

Each instruction still takes four stages, but after filling the pipeline an ideal core completes one instruction per cycle. Four independent instructions take seven cycles here instead of sixteen without overlap. This improves **throughput**, rather than reducing the latency of one instruction to one cycle. Dependencies, branches and competition for resources can introduce stalls or discard work, so ideal speed-up is not guaranteed.

Each core can operate its own pipeline while other cores execute separate threads in parallel. Pipelining also exists in single-core CPUs; it does not require an instruction's stages to move between cores. Multi-core gains depend on the OS distributing work and the program having parallel tasks. Cores may share RAM and caches, so access must be coordinated and shared copies kept consistent.

A1.1.7 Internal and external secondary memory storage

Secondary storage retains programs and data without power. It usually offers greater capacity at lower cost per bit than primary working memory, but slower access. Internal storage is fitted within a device; external storage connects through an interface or network. The same HDD/SSD technology can be used in either form.

Type	How it works / strengths	Limitations and appropriate use
HDD	Magnetized regions on rotating platters; a moving read/write head accesses them. Large capacities and usually low cost per GB.	Mechanical seek/rotation delays; vulnerable to shock when operating. Suitable for bulk files, backups and large archives.
SSD	NAND flash retains electrical charge; a controller manages reading, writing and wear. No moving parts, low access latency and shock resistance.	Usually higher cost per GB; flash has finite write/erase endurance. Suitable for OS/application drives and fast portable storage. Actual speed depends on interface and device.
eMMC	Flash and controller in a compact package, commonly soldered to the motherboard.	Usually less expandable and slower than higher-performance SSDs. Suitable for embedded or inexpensive portable devices.

External options and scenarios

- **External HDD/SSD**: backup and transfer large files. HDD favours inexpensive capacity; SSD favours rapid access and portability. The connection can limit speed.
- **Optical discs/drives**: a laser detects changes in reflection along a track on a CD, DVD or Blu-ray disc and decodes the stored bits. Useful for distributing media, compatibility with older equipment or suitable archives; slower access and susceptibility to scratches are drawbacks. Recordable/rewritable media use different recording layers from pressed discs.
- **USB flash drives**: small removable NAND-flash devices for file transfer. Convenient, but easy to lose, with variable speed/endurance; protect confidential files.
- **Memory cards**: compact removable flash storage, such as SD/microSD, used by cameras and portable devices. Check interface compatibility and speed requirements; small size makes loss/damage easy.
- **Network attached storage (NAS)**: a network-connected device providing centralized shared files and backups, often using HDDs/SSDs. Multiple users can access it, but it requires administration, access controls and a network; network speed affects retrieval. RAID can tolerate some drive failures but does not replace a separate backup.

Choose storage by capacity, access speed, cost, portability, durability and the access/sharing requirements of the scenario, rather than memorizing device-specific capacities or transfer rates.

A1.1.8 Compression

Compression reduces the number of bits needed to represent data, saving storage and transmission bandwidth/time. Compression/decompression also uses processing time, and not all data compresses well.

- **Lossless**: the exact original can be reconstructed. Encodes statistical redundancy efficiently. Needed for programs, text, financial records and backups, where changed bits could change meaning. Examples include ZIP and PNG.
- **Lossy**: deliberately discards information, so the original cannot be reconstructed exactly. Often gives greater size reduction for images/audio/video by reducing less noticeable detail; can introduce visible or audible artefacts. Suitable when this quality trade-off is acceptable, such as streaming. Typical examples include JPEG images and MP3 audio.

Run-length encoding (RLE) is lossless. Replace each consecutive run with a count and value. For example, `AAAAAABBBBCC` becomes `6A4B2C`, assuming that count–symbol format is agreed. It helps with long

repeated runs, such as uniform areas in a simple bitmap. It can make non-repetitive data larger: `ABCD` becomes `1A1B1C1D`. Storage savings depend on how counts and values are encoded, not just how many printed characters appear.

Transform coding, used in lossy media compression, changes the representation of data to make information easier to compress. For a JPEG-style image example:

1. Divide image data into blocks and apply a transform such as the discrete cosine transform (DCT), representing each block by frequency coefficients rather than raw pixel values.
2. **Quantize** the coefficients: reduce their precision, often retaining less fine detail. This is the information-losing step; stronger quantization generally yields smaller files and poorer reconstructed quality.
3. Encode the remaining coefficients efficiently. The decoder reverses the coding and transform steps, reconstructing an approximation of the original.

The transform itself is not what necessarily loses information; quantization does. Know this process and the trade-off, rather than the DCT mathematics. RLE exploits repeated adjacent values; transform coding suits complex media where exact reconstruction is unnecessary.

A1.1.9 Services in cloud computing

Cloud computing provides remote computing resources over a network, usually the internet. The service model determines which layers the provider manages and which the customer controls.

Model	Provider supplies/manages	Customer controls	Scenario and trade-off
SaaS	Hosted application, runtime, OS and infrastructure; application updates.	Account settings, permitted configuration, user access and their data.	A school uses hosted email or online documents. Least administration, but limited application customization.
PaaS	Application platform/runtime and underlying OS/infrastructure, with deployment services.	Own application code, data and supported platform settings.	A student deploys a web app on a managed platform. Faster development with less administration, but platform/language constraints.
IaaS	Virtual machines, storage and networking on provider-owned physical hardware.	Guest OS, software/runtime, applications, data and virtual-resource configuration.	An organization rents VMs for a custom server. Greatest control of these models, but more maintenance and security responsibility.

From SaaS to PaaS to IaaS, customer control/flexibility and administration generally increase. Resources can often be provisioned or scaled as demand changes, reducing upfront hardware purchases. However, availability depends on connectivity, provider capacity and service terms; recurring charges, data-transfer costs and dependence on a provider matter. No model is automatically cheapest or completely secure. Customers remain responsible for appropriate access and data handling even when the provider manages the application. SaaS may be accessed by a browser *or* an installed client; the service responsibilities define the model.

A1.2 Data representation and computer logic

A1.2.1 Principal methods of representing data

Binary (base 2) uses 0 and 1; **decimal/denary** (base 10) uses 0–9; **hexadecimal** (base 16) uses 0–9 and A–F, with A=10 through F=15. A digit's position gives its place value. A **bit** is one binary digit, a **nibble** is four bits and a **byte** is eight bits. Always identify the base when an answer could be ambiguous.

Binary place	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	128	64	32	16	8	4	2	1
Example bits	1	0	1	0	1	1	0	1

1. **Binary to decimal**: add the place values of the 1 bits. For the table above, $10101101_2 = 128 + 32 + 8 + 4 + 1 = 173_{10}$.
2. **Decimal to binary**: repeatedly divide by 2, record each remainder, and read remainders from last to first. Alternatively, subtract the largest available powers of 2: $173 = 128 + 32 + 8 + 4 + 1$, so the bits are 10101101_2. Include required leading zeros for a fixed width.
3. **Hexadecimal to decimal**: multiply digits by powers of 16 and add: $2F3_{16} = 2(256) + 15(16) + 3 = 755_{10}$.
4. **Decimal to hexadecimal**: repeatedly divide by 16 and read the remainders in reverse. $755 \div 16 = 47$ remainder 3; $47 \div 16 = 2$ remainder 15 (F); $2 \div 16 = 0$ remainder 2. Hence $755_{10} = 2F3_{16}$.
5. **Binary to hexadecimal**: group into nibbles from the right, padding the leftmost group with zeros if needed. $10101101_2 = 1010\ 1101_2 = AD_{16}$.
6. **Hexadecimal to binary**: replace each hex digit with exactly four bits, retaining the order. $2F3_{16} = 0010\ 1111\ 0011_2$. Hex is a compact, readable way of displaying binary values; it does not change the underlying stored bits.

Hex	0	1	2	3	4	5	6	7
Binary	0000	0001	0010	0011	0100	0101	0110	0111
Hex	8	9	A	B	C	D	E	F
Binary	1000	1001	1010	1011	1100	1101	1110	1111

A1.2.2 How binary is used to store data

Digital hardware distinguishes two physical states, such as voltage ranges or stored charge states, to encode 0 and 1. Bit patterns can represent different kinds of data; their **encoding and context** determine their meaning. For example, `01000001` can be unsigned integer 65 or character `A`. Standard formats let different systems interpret stored data consistently. More bits can represent more values/detail but require more storage and transfer time.

Integers

- An **unsigned n-bit integer** has range 0 to $2^n - 1$. For 8 bits, this is 0–255.
- **Two's complement** represents signed integers with range -2^{n-1} to $2^{n-1} - 1$. The most significant bit has weight -2^{n-1}, while the remaining bits have positive weights. For 8 bits, the range is -128 to 127.
- To encode a negative value at a fixed width, write its positive magnitude in binary, invert every bit, then add 1, retaining that width. For -13: $00001101 \to 11110010 \to 11110011$. Check: $-128 + 64 + 32 + 16 + 2 + 1 = -13$.

- Interpretation matters: 11110011 is 243 unsigned but −13 in 8-bit two's complement. 10000000 represents −128, not a separate sign followed by a magnitude of zero. A value outside the chosen range cannot be represented without changing the width/encoding.

Characters and strings

- A character encoding maps characters to numeric codes stored in binary; a string is an ordered sequence of encoded characters.
- **ASCII** is a 7-bit code with 128 values for English letters, digits, punctuation and control characters. `A` is decimal 65, commonly stored in a byte as `01000001`. So-called extended ASCII encodings are separate 8-bit extensions, not one universal ASCII standard.
- **Unicode** assigns code points for characters across many writing systems. An encoding such as **UTF-8** stores a code point using one to four bytes and preserves ASCII's codes. Unicode is not simply a fixed 16-bit code. Text storage depends on the encoding and characters; one visible character need not occupy one byte.

Images: a bitmap is a grid of pixels, each encoded with a colour value. **Resolution** gives the pixel dimensions (width × height). **Colour depth** is bits per pixel; b bits allow up to 2^b codes. In 24-bit RGB, each pixel uses 8 bits each for red, green and blue. A red pixel can be `11111111 00000000 00000000`. More pixels permit finer spatial detail; greater colour depth permits finer colour distinctions. Neither recreates detail already missing from the source.

$$\text{Uncompressed bitmap size (bits)} = \text{width} \times \text{height} \times \text{bits per pixel}.$$

A 640 × 480 image at 24 bits per pixel needs 7,372,800 bits, or 921,600 bytes of pixel data. File headers/metadata add storage; compression changes the actual size.

Audio: a microphone converts sound into an electrical analogue signal. An analogue-to-digital converter (ADC) **samples** its amplitude at regular intervals and **quantizes** each sample to an available numeric level. Values are stored in binary. The **sample rate** (Hz) is samples per second per channel; higher rates capture finer changes over time. The **bit depth** is bits per sample; more bits allow more amplitude levels and finer precision. Stereo has two channels.

$$\text{Uncompressed audio size (bits)} = \text{sample rate} \times \text{bit depth} \times \text{channels} \times \text{duration (s)}.$$

For 10 seconds at 44,100 Hz, 16 bits and two channels: 44,100 × 16 × 2 × 10 = 14,112,000 bits, or 1,764,000 bytes before headers. Without duration, the product describes *bits per second*, not file size.

Video: a sequence of image frames displayed at a **frame rate** (frames per second, fps), often synchronized with an audio track. Each uncompressed frame can be represented like a bitmap. Larger resolution, colour depth, frame rate or duration increases the raw data volume. For raw video without audio:

$$\text{Size (bits)} = \text{width} \times \text{height} \times \text{colour depth} \times \text{fps} \times \text{duration (s)}.$$

A 320 × 240, 24-bit, 25 fps clip lasting 4 seconds requires 184,320,000 bits, or 23,040,000 bytes for frames. Add audio separately if specified. Video compression reduces this volume by encoding redundancy within and between frames; stored files also contain information needed for decoding and synchronization.

Units and retrieval: divide bits by 8 for bytes. SI prefixes use powers of 1,000 (1 kB = 1,000 bytes; 1 MB = 10^6 bytes); binary prefixes use powers of 1,024 (1 KiB = 1,024 bytes; 1 MiB = 2^{20} bytes). Some classroom questions use KB/MB for binary multiples: follow any stated convention and show it in calculations. A decoder needs the agreed encoding and metadata (for example dimensions, sample rate or character encoding) to reconstruct the intended data. Compressed data must be decompressed before use; binary storage alone does not imply compression.

A1.2.3 Purpose and use of logic gates

A **logic gate** implements a Boolean function on binary inputs to produce a binary output. Conventionally 1 means true/on and 0 means false/off. Transistors arranged as switching circuits implement these functions. Combinations of gates form decision circuits, ALU arithmetic circuits and storage elements.

Notation: $A \cdot B$ or AB means AND; $A + B$ means inclusive OR; $\overline{A}$ means NOT; $A \oplus B$ means XOR. Boolean + is not ordinary addition: $1 + 1 = 1$. Evaluate brackets and complements carefully.

Gate	Standard symbol	Expression	Output is 1 when...
AND		AB	Both inputs are 1.
OR		$A + B$	At least one input is 1 (including both).
NOT		$\overline{A}$	Its single input is 0.
NAND		$\overline{AB}$	The AND result is 0.
NOR		$\overline{A + B}$	Both inputs are 0.
XOR		$A \oplus B$	The two inputs differ.
XNOR		$\overline{A \oplus B}$	The two inputs are the same.

The small circle (bubble) on a symbol means inversion. XOR has an additional curved input line compared with OR; XNOR adds an output bubble to XOR. The following table summarizes every two-input gate; NOT separately maps $0 \to 1$ and $1 \to 0$.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

Applications: an AND condition grants access only if a badge is valid *and* a code is correct. OR can trigger an alarm if smoke *or* excessive heat is detected. XOR detects unequal bits and supplies the sum output of a half adder; AND supplies its carry. XNOR detects equality. NAND/NOR gates can each be combined to implement other gates; feedback arrangements can hold a state in storage circuits. Know the gate's logical behaviour and application, rather than transistor-level circuit designs.

A1.2.4 Constructing and analysing truth tables

A truth table lists an output for *every* possible input combination. For n independent binary inputs, use 2^n rows. Enumerate input rows systematically, then add columns for intermediate results in the order required by the expression/diagram.

Worked problem: a security alarm is active when either motion (M) or an open door (D) is detected, provided the system is not in maintenance mode (T). Define 1 as the named condition being true. The expression is

$$X = M + D, \qquad Z = \overline{T}, \qquad Q = (M + D)\overline{T}.$$

M	D	T	$X = M + D$	$Z = \overline{T}$	$Q = XZ$
0	0	0	0	1	0
0	0	1	0	0	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	0	0
1	1	0	1	1	1
1	1	1	1	0	0

To derive an expression from a table, write an AND term for each row whose output is 1 (complement any input that is 0), then OR those terms. Here:

$$Q = \overline{M}D\overline{T} + M\overline{DT} + MD\overline{T} = (M + D)\overline{T}.$$

Two expressions or circuits are equivalent only if their outputs match for *all* input rows. A matching result for one input does not prove equivalence.

Boolean algebra for simplification

Law	**OR form**	**AND form**
Identity	$A + 0 = A$	$A \cdot 1 = A$
Domination	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotent	$A + A = A$	$AA = A$
Complement	$A + \overline{A} = 1$	$A\overline{A} = 0$
Commutative	$A + B = B + A$	$AB = BA$
Associative	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
Distributive	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan	$\overline{A + B} = \overline{A}\,\overline{B}$	$\overline{AB} = \overline{A} + \overline{B}$

Involution/double negation: $\overline{\overline{A}} = A$. In De Morgan's laws, complement each operand *and* interchange AND/OR. Simplify with named laws, for example:

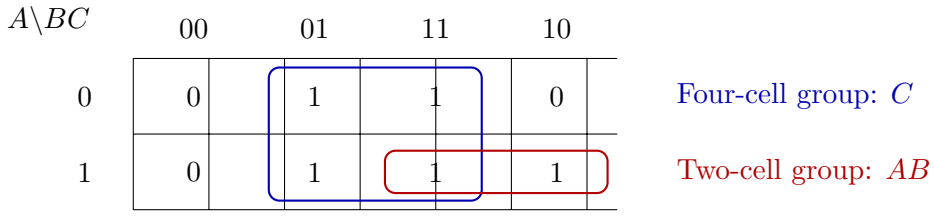
$$AB + A\overline{B} = A(B + \overline{B}) = A \cdot 1 = A.$$

Fewer terms/gates can reduce hardware complexity, cost and propagation delay, while preserving the truth table.

Karnaugh maps (K-maps) arrange truth-table outputs so adjacent cells differ in exactly one input bit. For paired labels use Gray-code order $00, 01, 11, 10$, not ordinary counting order. A two-input map has four cells; three inputs have eight; four inputs have sixteen.

1. Copy the output for each input combination into its labelled cell.
2. Group 1s into rectangular blocks containing powers of two cells $(1, 2, 4, 8, \ldots)$. Include no 0s. Diagonal cells are not adjacent. Opposite edges are adjacent, so groups can wrap horizontally/vertically; corners can form a group.
3. Cover every 1, using large groups and avoiding redundant groups. Groups may overlap if helpful.
4. For each group, keep only variables constant throughout it: use the variable when it is always 1 and its complement when always 0. Omit changing variables. AND the retained literals; OR the group terms.

Worked three-input map: suppose $Q = 1$ for $ABC = 001, 011, 101, 110, 111$ and 0 otherwise.



The four 1s in columns 01 and 11 have $C = 1$, while A and B vary, giving C. The two 1s at $A = 1$, columns 11 and 10, have $A = B = 1$, while C varies, giving AB. Their shared cell may belong to both groups. Hence $Q = C + AB$.

Wrap-around example (four inputs):

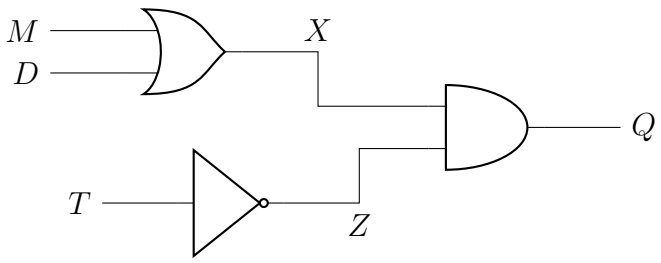
$AB\backslash CD$	00	01	11	10
00	1	0	0	1
01	0	0	0	0
11	0	0	0	0
10	1	0	0	1

The four corner 1s are adjacent across both edges and form a group of four. A and C vary; $B = D = 0$, so $Q = \overline{B}\,\overline{D}$. Never group these as a diagonal: it is their edge adjacency that makes the group valid.

A1.2.5 Constructing logic diagrams

Translate a problem description into defined inputs, an output condition and a Boolean expression. Break the expression into gates in evaluation order, using standard symbols; connect outputs of earlier gates to the required inputs of later gates. Label external inputs, intermediate signals where helpful and the final output. A dot marks a wire junction; crossing wires without a dot are not connected.

Worked diagram: implement the alarm $Q = (M + D)\overline{T}$ from A1.2.4.



Check a path explicitly: $M = 0, D = 1, T = 0$ gives OR output 1, NOT output 1 and AND output 1, so the alarm activates. Check all eight rows to validate the complete circuit.

To read a diagram, write the expression at each gate's output and substitute towards the final output. To simplify a diagram, derive its expression, apply algebra or a K-map, draw the reduced circuit and compare truth tables. For example, $AB + A\overline{B} = A$ needs only the signal A at the output; the gates are logically redundant. Do not substitute AND for OR, drop an inversion bubble, or assume XOR means inclusive OR.

A1.3 Operating systems and control systems

A1.3.1 Role of operating systems

An **operating system (OS)** manages hardware resources and provides services/interfaces for applications and users. It coordinates competing demands for CPU time, memory, storage and devices, maintaining efficient, fair and protected operation.

The OS **abstracts hardware complexity**: applications can request operations such as opening a file or printing without knowing a drive's physical layout or a printer's device-specific commands. The OS and device drivers handle those details. For example, a text editor requests that a file be saved; the OS checks access rights, manages its storage and communicates with the storage device. This gives applications a consistent interface and helps prevent them interfering with one another. A GUI is a user-facing part of a system; it is not the entire OS.

A1.3.2 Functions of an operating system

A **program** is a stored set of instructions; a **process** is an instance of a program in execution, with a state and allocated resources. A **thread** is a sequence of execution within a process; threads in the same process share resources such as its memory. The OS tracks processes and their resource use while supporting the following functions.

Function	What the OS does and why
Memory management	Allocates/releases RAM and protects address spaces so processes cannot freely overwrite each other or the OS. Virtual memory maps a process's virtual addresses to physical RAM; some pages may be kept on secondary storage and loaded when needed. This supports larger address spaces but disk transfers are much slower than RAM access.
File system	Organizes named files/directories, tracks their storage locations and free space, handles creation/read/write/deletion and enforces permissions. Applications use files rather than managing raw disk locations.
Device management	Uses drivers to communicate with specific hardware through a standard interface. Coordinates I/O requests, handles errors and completion events, and can buffer or queue output, such as print jobs.
Scheduling	Allocates CPU time among ready processes/threads, balancing responsiveness, fairness, priority and efficient use. A waiting I/O operation need not leave the CPU idle if another task is ready.
Security	Authenticates users, authorizes resource access and isolates/protects processes. Permissions, supported encryption, updates and security monitoring help protect confidentiality, integrity and availability.
Accounting	Measures resource use such as CPU time, memory, storage and network traffic by process/user. Supports monitoring, quotas, capacity planning, auditing and, where appropriate, charging. It is more than financial bookkeeping.
Graphical user interface	Provides visual interaction through windows, icons, menus and controls. Translates user actions into requests for system/application operations.
Virtualization	Supports software-created computing environments. A hypervisor allocates physical CPU, RAM, storage and I/O among virtual machines, which can run separate guest OSs. This improves utilization and isolation, but sharing adds overhead and isolation is not absolute.
Networking	Provides communication services/protocols, manages network interfaces through drivers, and supports connections, addressing and access controls so applications can exchange data. Detailed protocol layers belong to A2.

System integrity and background operations: integrity means the system and its data remain correct, reliable and protected from unauthorized/unintended changes. Background tasks can check for updates, scan for malware, write logs, monitor resource use and maintain storage without direct user interaction. The OS schedules them so they do not monopolize resources, enforces permissions and memory protection, and records faults/security events. For example, an update service needs appropriate privileges, but an ordinary application must not freely overwrite OS files. Background tasks still consume CPU, RAM and I/O; they are managed alongside interactive work.

A1.3.3 Comparing approaches to scheduling

A **scheduler** selects which ready process/thread receives CPU time. **Pre-emption** allows the OS to suspend a running task, for example when its time slice expires or a higher-priority task becomes ready. A **context switch** saves its execution state and restores another's, costing time during which useful application work is not performed.

Approach	Mechanism / advantage	Limitation / suitable scenario
First-come, first-served (FCFS)	Runs ready tasks in arrival order without pre-empting them. A task holds the CPU until it finishes its CPU burst or blocks. Simple, with low scheduling overhead.	Short tasks can wait behind a long one (convoy effect), giving poor interactive response. Useful where simplicity and ordered batch processing matter.
Round robin (RR)	Each ready task receives up to a fixed time quantum in cyclic order; unfinished runnable tasks return to the back. Shares CPU access and supports interactive response.	Very short quanta increase context-switch overhead; very long quanta behave more like FCFS. Suitable for time-sharing/interactive systems.
Multilevel queue	Separates tasks into fixed queues by type/class, each with its own policy (for example RR for interactive work and FCFS for batch work). Schedules between queues by priority or allocated shares.	More complex to configure. Strict preference for higher queues can starve lower queues. Useful for separating system, interactive and batch workloads.
Priority	Selects the highest-priority ready task. Can be pre-emptive or non-pre-emptive. Important work receives preference.	Low-priority tasks may starve; ageing can raise the priority of long-waiting tasks. Useful when some tasks are more urgent, provided policy and timing are suitable.

Worked comparison: P1, P2 and P3 arrive together in that queue order and need 5, 2 and 1 time units respectively. Ignore switching overhead and I/O.

- FCFS: P1 runs 0–5, P2 runs 5–7, P3 runs 7–8. The short P3 waits 7 units before its first execution.
- RR with quantum 2: P1 runs 0–2, P2 runs 2–4 (finishes), P3 runs 4–5 (finishes), P1 runs 5–7 and 7–8 (finishes). P3 starts earlier, but P1 finishes later.

Both complete eight units of work here; the policy changes response and completion times. Compare scheduling using the given workload, fairness, waiting/response time, throughput and switching overhead. No policy is always fastest. Ordinary FCFS waiting behind finite CPU bursts is not the same as indefinite starvation, and multilevel queue scheduling does not by itself move tasks between queues.

A1.3.4 Evaluating polling and interrupt handling

Polling: the CPU executes checks of a device/status flag to discover whether service is required, repeatedly or at scheduled intervals. If no event is present, the check still costs processing time. With periodic checks, an event just after one check waits until the next.

Interrupt handling: a device or software raises a request for attention. When an enabled interrupt is accepted, the processor saves the necessary execution context and runs an **interrupt service routine (ISR)**. The handler services/acknowledges the event, then execution can resume by restoring context (or the OS may schedule other ready work). Interrupt priorities, masking and time spent in other handlers affect response; the response is not literally instantaneous.

Factor	Polling	Interrupts
Event frequency	Repeated empty checks waste work for rare events. Frequent available data can justify checking/batching, with less per-event interrupt overhead.	Efficient for occasional events because the CPU works or sleeps between them. Extremely frequent interrupts can overwhelm useful work (an interrupt storm).
CPU overhead	Cost depends on checking frequency and how many devices are checked; busy-waiting ties up CPU time.	Saving/restoring state and executing handlers costs CPU time per interrupt; batching can reduce this cost.
Battery or mains	Frequent checks/wake-ups use energy. Mains power reduces the battery constraint, but wasted CPU time still matters.	Allows sleep between events, often conserving battery power; repeated wake-ups still cost energy.
Predictability	A known periodic event can be checked at suitable times. A fixed polling schedule can give a calculable maximum detection delay.	Useful for unpredictable events because service is requested when needed. Interrupt priority and handler duration must be controlled.
Controlled latency	Detection can be bounded by the polling interval plus checking/service time if scheduling guarantees hold; intervals must be short enough for the requirement.	Often prompt, but latency depends on masking, priorities and contention. A real-time system needs a justified bound, not merely "uses interrupts".
Security	Validate status/data; excessive polling can consume resources, and infrequent checking can delay detection of problems.	Untrusted event floods can cause denial of service; validate input and control event rates/priorities. Neither method is inherently secure.

Scenario judgments: for keyboard/mouse input, event-driven handling lets applications do other work between user actions (the device link itself may involve polling). Disk-completion interrupts let the CPU work while I/O is in progress. A battery-powered sensor can wake on an interrupt rather than check continuously. A busy network interface may combine interrupts with polling/batching to cope with high traffic. In a real-time controller, use a polling interval or interrupt design that satisfies the response deadline; either approach must be assessed against its timing and overhead.

A1.3.5 Multitasking and resource allocation (HL only)

Multitasking lets multiple processes make progress over overlapping periods. On one core, the OS rapidly switches among runnable tasks; on multiple cores, some can execute in parallel. **Concurrency** therefore does not always mean simultaneous physical execution.

- **Task management and scheduling**: the OS tracks process identity, priority, state and saved context. A ready task can run; a blocked task waits for an event/resource and does not become runnable merely because time has passed. The scheduler assigns CPU time and a context switch preserves each task's place.
- **Memory allocation**: allocate/protect each process's memory, reclaim it when appropriate and use virtual memory to map pages into RAM. Excessive paging can cause thrashing: much time is spent transferring pages rather than executing tasks.

- **Device/file allocation**: drivers, queues and access controls coordinate shared devices and files. For example, queued print jobs prevent several processes from sending interleaved output to one printer.
- **Resource monitoring**: usage accounting and limits help prevent one process consuming all CPU, memory or storage. Balance fairness, urgent work, useful throughput and switching/coordination costs.

Resource contention occurs when tasks compete for a resource with insufficient capacity for all requests. Access may need to be queued or protected using a lock. For example, two threads updating the same shared value must coordinate their read–modify–write operations to avoid losing an update. Locks protect shared work but can introduce waiting.

Deadlock occurs when tasks cannot proceed because each is waiting for a resource held by another in a cycle. For example, P holds the printer lock and waits for a file lock, while Q holds the file lock and waits for the printer lock. Neither can finish and release what the other needs. A consistent order for acquiring resources can prevent this cycle. A system may avoid unsafe allocations or detect a cycle and recover by terminating/restarting a task and releasing its resources. Recovery may lose work, and some resources cannot safely be taken away. Merely giving a deadlocked task more CPU time does not supply the missing resource. Distinguish deadlock from **starvation**, where a task is repeatedly denied resources while others continue making progress.

A1.3.6 Control system components (HL only)

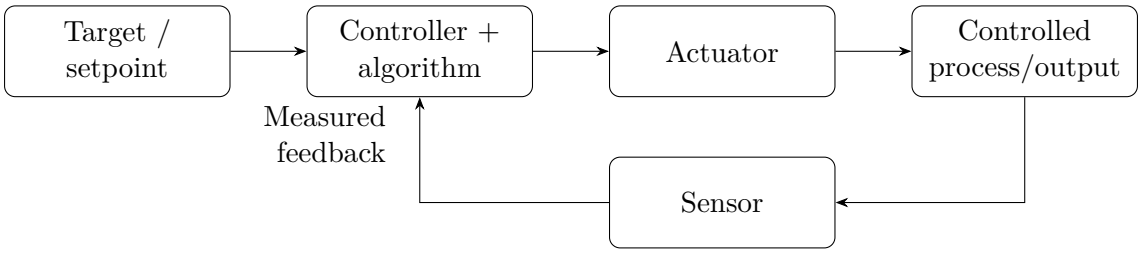
A **control system** uses inputs and a control algorithm to direct the behaviour of a device or physical process.

- **Input**: target/setpoint, user commands and/or measurements from sensors.
- **Process**: the controller applies the control algorithm to those inputs and determines a control action, such as comparing measured temperature with a target.
- **Output**: the actuator's action and resulting change in the controlled system, such as a heater being switched on and the room warming.
- **Feedback**: measurements of the actual output/state returned to influence the next control action. Negative feedback corrects a deviation from the target.

Component	Role and example
Controller	Processes inputs and sends control signals. For example, a microcontroller compares a temperature reading with a target.
Sensor	Detects/measures a physical quantity and produces a usable signal. Examples: thermistor for temperature, light sensor, pressure sensor.
Actuator	Converts a control signal into a physical action. Examples: motor moves a door; solenoid valve changes water flow; heater supplies heat.
Transducer	Converts one form of energy/physical quantity into another. A microphone converts sound into an electrical signal; a motor converts electrical energy into movement. Sensors and actuators can be input/output transducers, rather than transducers being a compulsory separate box.
Control algorithm	Rules/calculations determining the output from inputs and targets. For example, turn heating on below a lower threshold and off above an upper threshold to avoid rapid switching. Detailed PID mathematics is not needed here.

Open-loop: performs its action without using output feedback to adjust it. A timer waters plants for five minutes regardless of whether the soil is already wet. Simple and inexpensive, but cannot correct for changing conditions or errors in the result.

Closed-loop: uses measured output feedback to adjust control. Soil-moisture readings determine when watering starts/stops. It can respond to disturbances, but adds sensor/controller cost and depends on accurate measurements. Having any sensor does not automatically make a system closed-loop: the measurement must feed back the controlled result.



Closed-loop model. Removing the output-measurement feedback path gives an open-loop model.

A1.3.7 Real-world uses of control systems (HL only)

In an explanation, connect the **input, decision, action and feedback** to the desired outcome; naming components alone is insufficient.

- **Autonomous vehicle**: cameras, distance and speed sensors provide observations; the controller determines steering/braking actions; actuators turn steering and apply brakes. Updated speed/position measurements feed back the resulting motion so control can be corrected. Obstacles and changing road conditions require continual reassessment.
- **Home thermostat**: a temperature sensor measures room temperature; the controller compares it with the user's target; a relay/valve controls heating. New temperature readings feed back the heating result. This reduces deviation from the desired temperature; separated on/off thresholds avoid rapid switching near one boundary.
- **Elevator**: floor requests, position sensors and door sensors are inputs. The controller selects movement/stops and checks door conditions; motors move the car and doors. Position/door feedback lets it stop at the requested floor and avoid moving with an unsafe door state.
- **Washing machine**: the selected program and water-level/temperature sensors inform the controller. It operates inlet valves, heater, pump and drum motor. Feedback stops filling at the target level and regulates heating. A timed substage may be open-loop even though other parts use feedback.
- **Traffic signals**: a fixed-time light sequence is open-loop with respect to traffic demand. Vehicle detectors can instead feed traffic measurements to a controller that adjusts phase durations. Light-control outputs implement those phases; safe sequencing prevents conflicting flows from receiving permission together.
- **Irrigation**: moisture sensors report soil conditions; the controller compares them with the required range and operates valves/pumps. Repeated readings show the effect of watering and stop further supply when sufficient, reducing water waste compared with a fixed timer.
- **Home security**: motion/door/smoke sensors report events; the controller applies rules, including whether the system is armed; sirens, locks and notifications respond. A sensor-to-alarm chain alone is not necessarily closed-loop control of the alarm's effect; specify what output is measured and used if claiming feedback.
- **Automatic door**: approach sensors request opening; the controller drives a motor; position and obstruction sensors report door state and prevent unsafe closure. Feedback supports stopping at the open/closed limits and reopening or stopping when obstructed.

A1.4 Translation (HL only)

A1.4.1 Evaluating interpreters and compilers

Translation converts source code into a form that can be executed. Machine code contains instructions for a particular processor architecture. **Bytecode** is an intermediate instruction format for a virtual machine/runtime. Translation strategies belong to implementations, so one language can have interpreted, compiled or hybrid implementations.

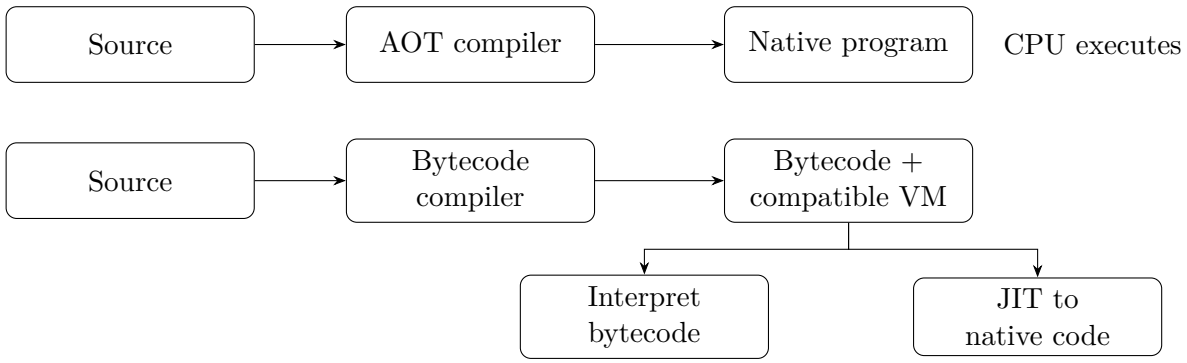
1. **Source interpretation**: an interpreter analyses source and carries out its operations during execution, without producing a separate native executable as its main output. The simple classroom model is "translate and execute one statement at a time", but an implementation may parse a whole unit before executing it. No separate user-visible build step gives a short edit–run feedback cycle; ongoing interpretation generally adds execution overhead.

2. **Ahead-of-time (AOT) compilation**: a compiler analyses source before execution, checks syntax and applicable static semantic rules, and generates target code, often with optimizations. For a native executable, machine/object code is linked with required code/libraries as appropriate. The resulting program can run without the original compiler, although it may still require libraries/runtime support. Source changes normally require rebuilding affected code.

3. **Bytecode interpretation**: source is compiled into an intermediate format; a compatible VM/interpreter executes bytecode operations. This avoids repeatedly processing the original source syntax and allows the same compatible bytecode to run on different systems with suitable runtimes. The bytecode is not normally executed directly by the CPU. CPython is an example of a source-to-bytecode interpreter implementation.

4. **Just-in-time (JIT) compilation**: a runtime compiles code to native machine code during execution, often identifying frequently executed sections (hot spots). It can optimize using runtime information and reuse the generated code during that execution. Warm-up, compilation time and memory overhead can be worthwhile for repeatedly executed work. A runtime such as a JVM can combine interpretation and JIT compilation; these are not mutually exclusive approaches.

Mechanics in brief: lexical analysis identifies tokens; parsing checks their grammatical structure; applicable semantic checks examine meaning such as type use. Code generation creates the target representation and optimization aims to improve efficiency without changing required behaviour. These steps may occur in different translation strategies. A translator cannot in general prove that the program solves the intended problem.



Two common paths. A VM can interpret some code and JIT-compile other code; not every runtime uses both.

Error detection

- Compilation can report syntax errors and applicable static errors before an executable/bytecode unit is produced. For example, a missing delimiter or a statically invalid operation may be rejected without running that code.

- An interpreter may also detect syntax errors before execution when parsing a module. Other errors emerge only on executed paths, such as dividing by a value that becomes zero. Bytecode compilation/verification can catch some errors before the bytecode runs.
- Runtime errors and logic errors can occur with *all* strategies. Successfully compiling does not establish correct output, safe data or freedom from crashes. The error checks available depend on the language/runtime, not simply whether it is called a compiler or interpreter.

Approach	**Translation / execution**	**Portability / deployment**
Source interpreter	Typically short edit–run cycle; analysis/execution overhead remains while running. Parsing may occur first.	Source can run with a compatible interpreter, libraries and OS services. Platform-specific dependencies can still limit portability.
Native AOT compiler	Build work occurs before execution; optimized native code often runs efficiently without runtime translation. Rebuild after relevant changes.	Executable targets an OS/architecture; another target generally requires rebuilding. Source can still be portable when compatible tools/libraries exist.
Bytecode interpreter	Initial source-to-bytecode compilation, then interpretation overhead during execution; avoids repeatedly parsing source.	Same bytecode can run with a compatible VM/version and required libraries on different hosts. VM implementations are platform-specific.
JIT runtime	Compilation and optimization occur during a run, using time and memory; hot code can then execute efficiently. Warm-up may dominate a short task.	A compatible runtime can produce appropriate native code on each host. Portable input code does not make the generated native code portable.

There is no fixed universal ranking for translation time, memory use or execution speed: program, optimization, runtime and workload matter. The key comparison is *when* translation happens, what is executed, and what support each target system needs.

Required application scenarios

- **Rapid development and testing**: a short data-processing script or prototype benefits from an interpreter's quick edit–run cycle and interactive feedback. Runtime overhead may be acceptable; test paths are still needed to expose errors. Compiled tools can also offer fast incremental builds, so justify the choice for the stated need.
- **Performance-critical application**: a simulation engine or time-sensitive native application can benefit from AOT optimization and avoiding JIT warm-up/compilation pauses. Build time and separate target builds are trade-offs. AOT alone does not guarantee real-time deadlines; algorithm and system behaviour also matter.
- **Cross-platform development**: distribute compatible bytecode, for example a Java application, with a suitable VM on each target. Interpretation and/or JIT can run the same core program on different systems; JIT may suit long-running work. Check runtime versions, libraries and OS-specific features. Native builds per target are another option when needed.

A2 Networks

A2.1 Network fundamentals

A2.2 Network architecture

A2.3 Data transmission

A2.4 Network security

A3 Databases

A3.1 Database fundamentals

A3.2 Database design

A3.3 Database programming

A3.4 Alternative databases and data warehouses

A4 Machine learning

A4.1 Machine learning fundamentals

A4.2 Data preprocessing

A4.3 Machine learning approaches

A4.4 Ethical considerations

B Computational Thinking and Problem Solving

B1 Computational thinking

B1.1 Approaches to computational thinking

Scope and reference: Oxford, MacKenty and Stephenson (2025), pp. 317–334; IB Computer Science guide (first assessment 2027), B1.1.1–B1.1.4. These four numbered paragraphs cover B1. Computational thinking is a toolkit for formulating and solving problems; it can be used without writing a program. Application examples below also draw on the classroom material supplied by the student and are expressed as condensed study notes.

B1.1.1 Problem specification

A problem specification defines what needs solving, who needs the solution, the restrictions, and how success will be judged. Agree it with the intended users before designing the solution: otherwise a technically correct program may solve the wrong problem.

1. **Problem statement**: identify the affected users, the current difficulty and its consequences. State what and why, using precise evidence where available. "Improve school administration" is too vague; "staff must compare room-booking records manually, so clashes can be missed" identifies a specific problem.

2. **Constraints and limitations**: state boundaries such as budget, deadline, available hardware, supported software, connectivity and privacy requirements. Distinguish a restriction on development from a limitation of the solution: an offline prototype cannot synchronize bookings across multiple computers.

3. **Objectives and goals**: an objective describes the desired broad outcome; specific goals make it measurable and, where appropriate, time-bound. For example, improve booking reliability; during a two-week pilot, detect every clash in an agreed test set.

4. **Input specifications**: define data items, types, formats, valid ranges, sources and validation rules. For example, a room ID must reference an existing room; a start time uses a 24-hour time format.

5. **Output specifications**: define the result's content, format, presentation and recipient. For example, show a confirmation containing the room, date and time, or an error explaining the clash.

6. **Evaluation criteria (success criteria)**: define testable benchmarks, linked to the original problem and goals, and state how evidence will be collected. Consider effectiveness (solves the problem), accuracy (correct results), efficiency (time/resources), usability (users can complete tasks) and maintainability (changes can be made safely). "Easy to use" is vague; "four of five staff can create a booking unaided within two minutes" is testable.

Goals versus criteria: a goal is a desired result; a criterion supplies the pass/fail measure used to evaluate it. An input or output specification describes required data or behaviour; testing against it provides evidence of success.

Worked real-world problem: avoiding school room-booking clashes

Illustrative specification: the quantities below are proposed requirements, not measurements of an actual school.

- **Problem statement**: teachers book shared rooms through separate messages. Staff cannot reliably see existing reservations, creating the risk of two activities being assigned the same room at the same time. A central booking tool should detect conflicts before accepting a reservation.

- **Constraints and limitations**: build a prototype in four weeks using existing school computers and no paid services. Pilot it on one staff-operated computer with local storage; a shared version would require centralized storage and concurrency control. Store only staff identifiers needed for booking, and restrict access to authorized staff. Assume bookings begin and end on the same date and use one school time zone.
- **Objective and goal**: improve room allocation; by the end of a two-week pilot, prevent all overlapping bookings in an agreed test set and let staff view a room's daily bookings.
- **Inputs**: room ID (string from a maintained list), staff ID (non-empty authorized identifier), booking date (valid date in `YYYY-MM-DD`), start and end times (`HH:MM`, 24-hour format). Require $start < end$. Existing bookings supply room, date, start and end values for comparison.
- **Outputs**: a saved booking with a unique reference and a confirmation showing room/date/times; or a validation/conflict message. A daily schedule lists accepted bookings in start-time order. Rejected requests must not alter stored bookings.

Measurable evaluation criteria and evidence

1. Reject every overlap in tests covering identical times, partial overlaps and one interval containing another; accept adjacent intervals, different rooms and different dates. Compare actual results with predetermined expected results.
2. Reject an unknown room, missing staff ID, impossible date, malformed time and $end \leq start$; display a useful error for each test.
3. After saving and restarting, retrieve every accepted test booking exactly once, with unchanged details; verify rejected requests were not saved.
4. On the pilot computer with 1,000 stored bookings, each of 20 test requests returns a result within one second. Record timings and the test environment.
5. At least four of five staff testers create a valid booking and locate it in the daily schedule unaided within two minutes; observe and record task completion.
6. Change the room list in its configuration without changing the overlap-checking algorithm; rerun the relevant acceptance tests to check maintainability.

How a computer program solves it:

1. Read and validate the requested room, date and times.
2. Retrieve existing bookings for the same room and date.
3. For each booking, test ($newStart < existingEnd$) AND ($newEnd > existingStart$). If true, report a clash and reject the request. Treat intervals as $[start, end)$, so a booking ending at 10:00 does not clash with one starting at 10:00.
4. If no overlap exists, save the booking, assign its reference and output confirmation. Sort the daily schedule by start time when requested.

Why use a program? The input is structured, the conflict rule is precise, and the same comparisons are repeated for every request. A program applies the rule consistently, searches many records quickly, preserves bookings and makes the schedule easy to retrieve. Accurate input and a shared source of truth are still necessary. For simultaneous online requests, checking and saving must be protected as one atomic operation, so two users cannot both reserve the same free slot. Staff still decide priorities and resolve exceptional requests.

B1.1.2 Fundamental concepts of computational thinking

- **Decomposition**: break a complex problem into smaller, manageable subproblems with clear responsibilities and interfaces. Parts can be developed, tested and debugged separately; independent tasks can be assigned to different people. In the room-booking example, separate input validation, conflict checking, storage and schedule display. Dependencies still determine which tasks must happen first.
- **Pattern recognition**: identify similarities, recurring structures or trends in problems, data or solutions, then reuse or adapt a suitable solution. Every booking uses the same interval-overlap test, so one function can handle every room. Repeated demand for a room at lunchtime may also reveal a usage pattern. A pattern suggests a useful generalization; it does not by itself prove a cause.
- **Abstraction**: retain details relevant to a purpose and hide or omit others, producing a simpler representation or interface. Conflict checking needs room ID, date and times, but not wall colour or furniture brand. A `saveBooking` interface hides file-handling details from the conflict checker. Which details matter depends on the task: room capacity matters if capacity checking is added. Abstraction manages complexity; it does not automatically make programs smaller or faster.
- **Algorithmic design**: create a precise, ordered and unambiguous set of steps to solve a problem, with defined inputs, outputs and termination conditions. Use sequence, selection and iteration as required. For booking: validate input, loop through relevant reservations, reject if an overlap is found, otherwise save and confirm. The design can be expressed in plain language, pseudocode or a flowchart before implementation in a programming language.

The concepts work together and may be revisited in any order. Decomposition asks "what smaller tasks?", pattern recognition asks "what is similar?", abstraction asks "what details matter?", and algorithmic design asks "what exact steps?" A person could use the same ideas to organize a paper booking register without programming.

B1.1.3 Applying computational thinking to solve problems

Related reading: InThinking B 1.1.3: Applying computational thinking. The examples below explain both how each concept is applied and why it helps.

Software development — an online shop checkout

- **Decomposition**: separate basket calculation, address validation, stock reservation, payment and order confirmation. Clear interfaces allow a payment provider to be changed without rewriting basket calculations.
- **Pattern recognition**: basket items, saved addresses and order history all require authorized retrieval of a user's records; shared data-access and validation routines reduce duplicated logic.
- **Abstraction**: a payment API accepts an amount and a payment token and returns a result, hiding the provider's internal bank communication and processing.
- **Algorithmic design**: validate basket and address, reserve stock, request payment, record the result, then confirm the order. Handle failure explicitly: release reservations and, if a charge has already succeeded, arrange a refund or other compensating action. An external payment cannot simply be undone by rolling back a local database transaction. This ordering and failure handling reduce inconsistent orders and duplicate charges.

Data analysis — school attendance reporting

- **Decomposition**: import records, validate and clean them, group by student, calculate attendance, identify students requiring review, and generate the report.

- **Pattern recognition**: repeated absences on particular weekdays or in one tutor group suggest questions for staff to investigate. Correlation alone does not establish the reason for absence.
- **Abstraction**: replace thousands of individual marks with each student's ID, expected sessions, present sessions and attendance percentage. Keep the raw records available for checking.
- **Algorithmic design**: standardize IDs/dates/status codes, resolve duplicates and missing or invalid records under an agreed policy, then calculate $100 \times present/expected$ for each student with $expected > 0$. Flag values below an illustrative 85% threshold and display one decimal place. Compare the unrounded value with the threshold; handle zero expected sessions as "not applicable". Standardization should precede duplicate detection when inconsistent formats can hide duplicates. Automated summaries save repetitive work; staff review the context before acting.

Network security — detecting repeated failed logins

- **Decomposition**: separate event collection, parsing, counting, rule matching, alerting and logging. Detection rules can change without replacing the collection system.
- **Pattern recognition**: many failed SSH logins from one source in a short period suggest a brute-force attempt. Signature or anomaly detection can flag suspicious activity, but legitimate failures may produce false positives and unfamiliar attacks may be missed.
- **Abstraction**: show an alert containing source IP, target service, time window and failure count instead of making an analyst inspect every event. A network diagram similarly retains relevant devices, connections and trust boundaries.
- **Algorithmic design**: on each failure, discard events outside the previous 60 seconds, update the source's count, and alert if it exceeds 100. If policy permits, temporarily block the source, log the action and schedule review. Counts require timestamps and a defined window. Consistent automated checks help staff respond quickly, while review accounts for shared IP addresses and false positives.

Database design — a school library

- **Decomposition**: divide the design work into identifying entities and requirements, defining tables/relationships and constraints, then choosing physical storage and indexes. Separate Students, Books and Loans so each represents a clear entity or relationship; Loans references the student and book using keys.
- **Pattern recognition**: seeing student details copied into many loan records suggests redundancy and update anomalies. Examine functional dependencies to decide how to normalize the design; repeated values alone do not prove a table is incorrectly designed.
- **Abstraction**: represent students by ID, name and year group, and loans by borrower, book and dates. Exclude medical details because they do not serve this library's purpose. The schema describes logical data without exposing disk layout.
- **Algorithmic design**: to report overdue loans, combine Loans with Students and Books using their keys, select unreturned loans with a due date before today, and sort by due date. Defined rules produce a repeatable, readable report. SQL describes the required result; the database engine chooses a physical execution plan, which need not follow the written clause order.

Machine learning — an email spam classifier

Machine learning fits a model to examples so it can make predictions for new inputs, rather than requiring a hand-written rule for every case.

- **Decomposition**: separate labelled-data collection, training/validation/test splitting, preprocessing, feature extraction, training, evaluation and deployment. Fit preprocessing on training data to avoid leaking information from evaluation data.
- **Pattern recognition**: learn statistical associations between spam labels and features such as word frequency, link count and sender characteristics. The aim is generalization to unseen emails; overfitting occurs when a model learns training-specific detail that does not generalize.
- **Abstraction**: represent an email as a numerical feature vector, such as vocabulary frequencies and link count. In a trained linear or neural classifier, learned weights summarize relationships between those features and labels. This description is model-specific: some ML methods retain training examples, and models can memorize information.
- **Algorithmic design**: for a neural classifier, initialize weights, process training batches, calculate loss, use backpropagation to find gradients and update weights; repeat subject to a stopping rule based on validation performance. Evaluate the final model on untouched test data, including false positives and false negatives. At deployment, a chosen threshold (for example, spam probability > 0.85) routes messages to spam or inbox. The threshold trades missed spam against legitimate messages incorrectly filtered; learning and consistent classification reduce manual sorting.

B1.1.4 Tracing flowcharts for programming algorithms

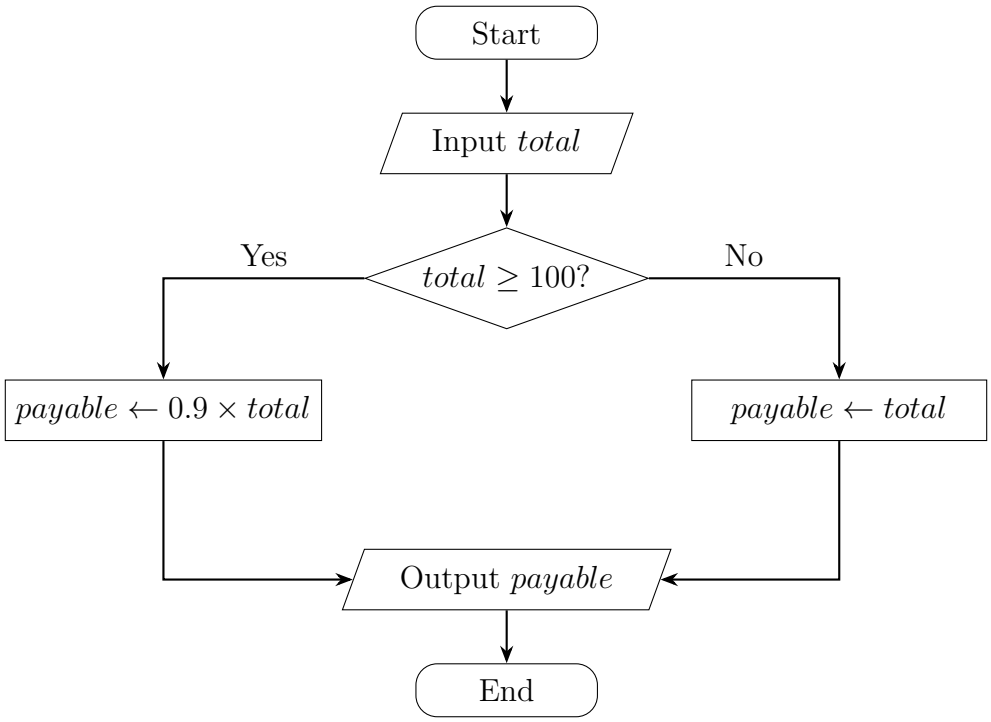
A **flowchart** represents an algorithm's control flow. Read from Start and follow arrowheads, usually downwards. At each decision, evaluate the condition using the *current* variable values, then follow only the matching branch. A loop returns to an earlier step. A trace table records changes in variables, tests and outputs; trace all iterations, including the final failed loop test.

Symbol	Shape	Meaning
Start/End	Start	Rounded rectangle or oval: entry or termination point.
Process/Operation	$x \leftarrow x + 1$	Rectangle: calculation, assignment or processing step.
Input/Output	Input x	Parallelogram: read input or display/write output.
Decision	$x > 0$?	Diamond: evaluate a condition; label the outgoing branches Yes/No or True/False.
Flowline		Arrow: direction and order of execution, including loop returns.
Connector	A	Small labelled circle: continue at the matching connector elsewhere on the same page; avoids long or crossing flowlines.

Tracing conventions: $\leftarrow$ means assignment, so $x \leftarrow x + 1$ replaces x with its old value plus one. A comparison such as $x = 5$ inside a decision tests equality. Record outputs only when an output symbol is reached. A variable retains its value until assigned again. Check that every route is defined and that a loop makes progress towards termination.

Example 1 — selection: school shop discount

Input a non-negative purchase total. If it is at least 100 currency units, apply a 10% discount; otherwise keep the original total. Output the amount payable. Assume valid input and use totals for which rounding is unnecessary in this example.



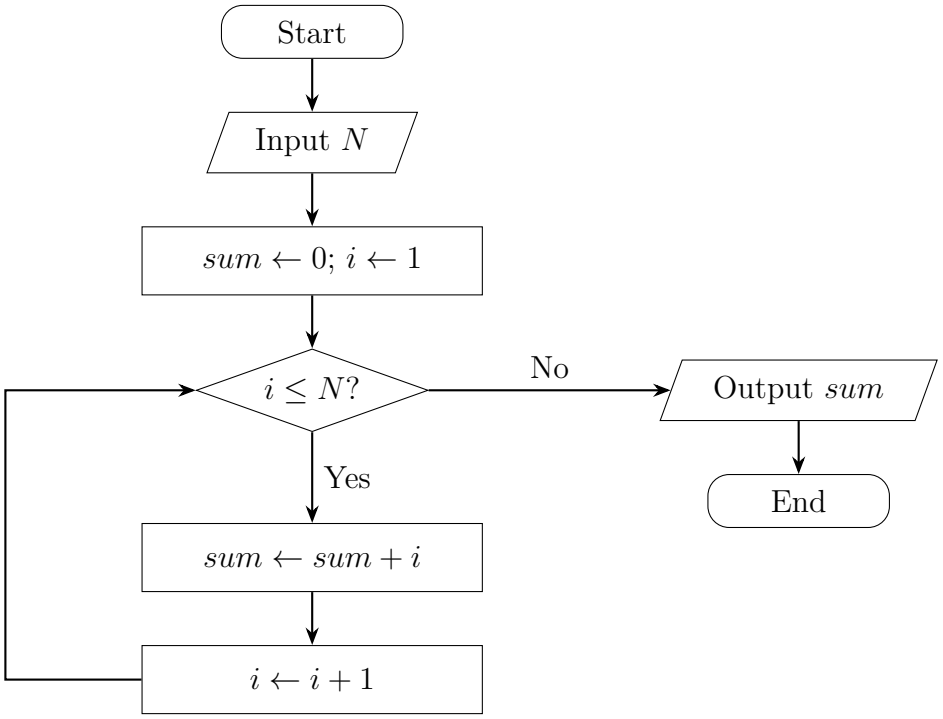
Trace table: independent runs

Input *total*	*total* ≥ 100?	Assigned *payable*	Output
80	False	80	80
100	True	90	90
150	True	135	135

For input 150, execution follows Start → input → decision (Yes) → discounted assignment → output 135 → End. The other assignment is skipped. The boundary input 100 takes the Yes branch because the comparison includes equality.

Example 2 — iteration: sum from 1 to N

Input a non-negative integer N. Add each integer from 1 to N and output the total. Initialize the accumulator before the loop; update both the accumulator and counter on each pass.



Trace for $N = 4$ ("after body" means after both assignments):

Test	i before	sum before	$i \leq 4$?	sum after body	i after body	Output
1	1	0	True	1	2	—
2	2	1	True	3	3	—
3	3	3	True	6	4	—
4	4	6	True	10	5	—
5	5	10	False	—	—	10

There are **four executions of the loop body and five condition tests**. On the final test, $i = 5$, so the No branch outputs 10. For the boundary input $N = 0$, the first test is false and the output is 0. Forgetting to increment i would cause a non-terminating loop for $N \geq 1$; using $i < N$ would omit N from the sum.

IA practice — adapt the room-booking example to a real user

1. Identify an accessible user, such as the teacher responsible for shared rooms. Ask how bookings are currently recorded, what goes wrong and which tasks take time. Record their actual needs rather than assuming the illustrative problem applies to them.

2. Agree a manageable problem specification and measurable success criteria with that user. Replace the example's proposed deadlines, data volumes and usability targets with requirements justified by the user's situation.

3. Explain how the program will transform inputs into outputs, apply all four computational-thinking concepts, and draw a flowchart for a key algorithm such as conflict checking. Explain why automation helps this user and compare it with their current method or an existing booking tool.

4. Plan normal, boundary and invalid test data for each criterion; for example, overlapping bookings, adjacent bookings and an end time before the start. Review a prototype with the user and evaluate the results against the agreed criteria, recording remaining limitations.

This is a planning exercise: a real project needs evidence from the actual user and testing, rather than invented interview findings or claimed results.

B1 completion check: B1.1.1 covers all six specification components and measurable criteria; B1.1.2 defines and illustrates all four CT concepts; B1.1.3 applies them to software development, data analysis, network security, database design and machine learning, as well as the room-booking problem and IA practice; B1.1.4 supplies all six standard symbols and two worked traces covering sequence, selection and iteration.

B2 Programming

B2.1 Programming fundamentals

B2.2 Programming constructs

B2.3 Data structures

B2.4 Programming algorithms

B2.5 File processing

B3 Object-oriented programming (OOP)

B3.1 Fundamentals of OOP for a single class

B3.2 Fundamentals of OOP for multiple classes

B4 Abstract data types (ADTs)

B4.1 Fundamentals of abstract data types

C Appendices

C0.1 My favourite operating system: Arch Linux (+ Hyprland)

C0.1.1 What it is

Arch Linux is a lightweight, rolling-release distribution built around the principle of user-centricity: you start from a minimal base and build up exactly the system you want, installing only the packages you actually need. On top of it I run Hyprland, a dynamic tiling Wayland compositor, which handles all the window management and visual effects. Together they are not a finished product you install, but a system you assemble and shape yourself.

C0.1.2 Why it is my favourite

- Customizability: nothing is assumed for me. Every package, service, kernel module and keybind is a deliberate choice, so the OS fits my workflow instead of the other way around. Hyprland is configured entirely through a plain-text file, so layouts, animations and rules can be tuned down to the pixel.
- Responsiveness and performance: the system ships with no bundled bloatware or background services, so boot times are short and idle resource usage is very low. On the same hardware, a minimal Arch + Hyprland setup feels noticeably snappier than a stock desktop environment.
- Rolling release: packages are updated continuously rather than frozen until the next major version, so I always have current kernels, drivers and toolchains without a disruptive reinstall.
- The AUR and community: the Arch User Repository gives access to an enormous range of community-maintained packages, and the Arch Wiki is arguably the best-documented Linux resource available, which makes troubleshooting a learning exercise rather than a dead end.
- Keyboard-driven workflow: as a tiling compositor, Hyprland automatically arranges windows with no manual dragging, which keeps my hands on the keyboard, reduces context switching and makes multitasking far faster.
- Learning and control: because I built the system, I understand how each layer fits together. Using Arch teaches real operating-system concepts — package management, the boot process, display servers, resource allocation — which connects directly back to the OS material in Section A.

A rolling-release, build-it-yourself system does come with trade-offs: upgrades occasionally break things and demand manual intervention, and there is no vendor support to fall back on. For me that trade-off is worth it — the control and understanding I gain outweigh the convenience I give up.